Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency

Anonymous Authors

July 2026

Abstract

Joint-embedding predictive architectures (JEPAs) learn world models that predict in a compact latent space rather than in pixels, reducing the pressure to model nuisance appearance. Yet this reduced pressure is not a guarantee: a visual perturbation that leaves the task state unchanged can still alter the encoded history and every prediction built on it, while indiscriminate invariance can erase state distinctions that determine the appropriate action. Bisimulation makes the required selectivity precise: two observations should be treated as the same state only when their action-conditioned consequences agree. Following this intuition, we introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic that encodes a clean history and a visually perturbed view of it, rolls both representations forward under the same action sequence, and measures the divergence between their predicted futures. We prove that this divergence bounds the perturbation-induced change in multi-step prediction error and planner cost. At the checkpoint level, two complementary measures form a screen: the Invariance Radius captures how far views of the same state spread after prediction, and the Distinction Rate checks that different states remain distinguishable. Experiments on four visual control tasks and two model families show that pairwise ACPC predicts these changes in practice and that the ACPC-based screen transfers to new tasks and visual shifts.

Keywords: visual world models; predictive consistency; action-conditioned rollouts; visual invariance; checkpoint diagnostics.

Anonymous reproducibility note: code and data availability follow the venue's double-blind supplemental policy.

1 Introduction

World models support control by predicting how candidate actions change the environment [1, 2]. Joint-embedding predictive architectures (JEPAs) instead formulate prediction in representation space rather than reconstructing pixels [3, 4, 5]. This can reduce pressure to preserve nuisance image detail [6, 7], but latent prediction alone does not establish robustness. Noise, blur, or reduced resolution can leave the underlying state unchanged while altering the encoded history and every prediction made from it. A robust model should ignore appearance changes that preserve the state, but it must retain state distinctions that can change the appropriate action. Bisimulation formalizes this requirement: two observations may represent the same abstract state only if, under every action, they have equal rewards and induce the same transition probabilities over bisimulation equivalence classes [8, 9]. It locates the test of sameness in action-conditioned consequences rather than in appearance—the yardstick our diagnostic adopts—although we do not test reward or transition equivalence.

What matters for control is how a visual discrepancy evolves under the actions considered by the planner. A planner does not act on an encoder output in isolation: it rolls that representation forward under candidate actions and ranks the predicted outcomes. A small encoder discrepancy may grow during this computation, while a larger one may contract before it reaches the planner. Encoder distance therefore cannot answer the central question: after applying the same actions, how far apart are the predicted futures of two views of the same history?

Action-Conditioned Predictive Consistency (ACPC) answers this question, as summarized in Figure 1. ACPC encodes a clean history and a visually perturbed view of that history, rolls both forward under one action sequence, and measures the distance between their multi-step predictions. It evaluates a frozen checkpoint and is not a training objective.

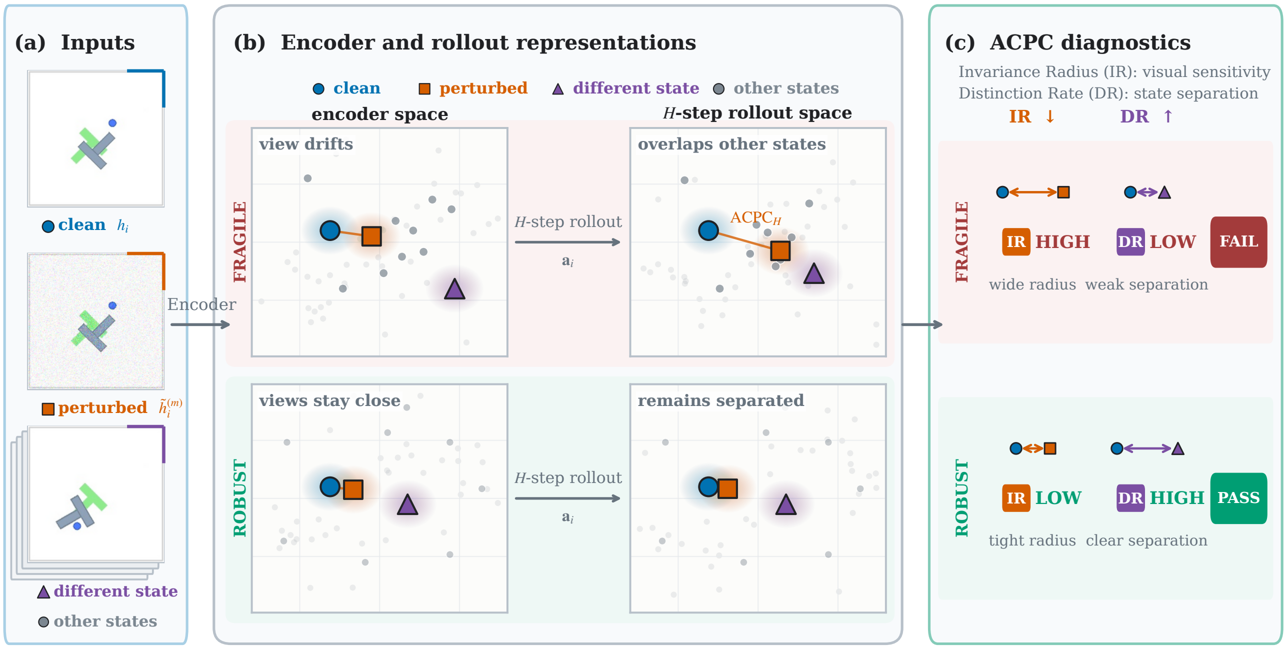

Figure 1: Overview of the Action-Conditioned Predictive Consistency (ACPC) diagnostic. **(a)** Inputs are logged PushT observations: a clean history, a Gaussian-perturbed view of the same history, and a nearby history with a different task state, shown for illustration and not necessarily the neighbor selected by the DR protocol. Stacked outlines indicate additional logged histories. **(b)** Schematic latent-space comparison of a fragile (top) and a robust (bottom) checkpoint. The frozen encoder embeds each input, and both branches are rolled out for H steps under the shared action sequence $\mathbf{a}_i$; ACPC measures the distance between the clean and perturbed predicted trajectories. Under the fragile checkpoint, the perturbation-induced spread grows during the rollout and overlaps other states; under the robust checkpoint, the paired views remain close while different states remain separated. Positions and neighborhoods are schematic and do not enter any reported metric. **(c)** Checkpoint-level summary of each row. The Invariance Radius (IR) aggregates normalized ACPC over perturbation draws and anchors and is reported relative to the unaugmented checkpoint; the Distinction Rate (DR) is the fraction of selected different-label pairs whose rollout distance exceeds the raw IR by a fixed margin. A checkpoint passes the screen only if relative IR is low and DR is high; passing is specific to the evaluated visual shift and state-coordinate labels and does not certify robustness.

A low ACPC value is not sufficient: a constant representation would make every paired rollout identical. We therefore summarize each checkpoint with two complementary measures. The *Invariance Radius* (IR) is a high quantile of normalized ACPC over logged histories; lower is better. The *Distinction Rate* (DR) is the fraction of nearby pairs with different state-coordinate labels whose rollout distance exceeds raw IR by a fixed margin; higher is better. DR applies only to the supplied coordinate labels.

ACPC also connects this measurement to downstream quantities. When a logged common future is available, an exact samplewise inequality bounds the perturbation-induced change in multi-step prediction error (Proposition 1). For squared latent goal costs, candidate-specific ACPC bounds every candidate's cost movement. Together with the clean cost margins, these bounds give sufficient conditions for preserving the fixed-pool winner or elite set (Proposition 2). These results bound the effect of a visual perturbation. They are deterministic, samplewise bounds on the evaluated predictions and require no distributional or global smoothness assumptions. They do not assert that either prediction is accurate or that a certified planner obtains higher simulator return.

We evaluate the diagnostic on four visual control tasks, using LeWM [10] as the primary JEPA world model and PLDM as a second model family (Section 4.7). At the pair level, on LeWM's unaugmented checkpoints, eight-step ACPC under recorded actions predicts perturbation-induced error drift with $51.3 \pm 3.5\%$ lower cross-validated MAE than the strongest same-horizon destroyed-action control (Section 4.4). Planner-horizon ACPC improves cross-task prediction of CEM selection regret by $15.2 \pm 2.0\%$, with an improvement in all 12 task–run evaluations (Section 4.5). At the checkpoint level, a joint IR–DR rule selected on two or three tasks places the first accepted checkpoint within one augmentation level of recovery on the remaining tasks (Section 4.6). Under blur and resize, the joint score agrees with the predefined success criterion on 22 of 24 checkpoint pairs at the tested severities (Section 4.8). We apply the same procedure to PLDM, choosing thresholds only from PLDM tasks.

The contributions are:

1. **A paired-rollout diagnostic** (Section 3). ACPC measures how a state-preserving visual change affects a frozen model's prediction under fixed actions; IR summarizes same-history sensitivity, and DR checks that selected state-coordinate distinctions remain separated.
2. **Exact links to prediction and planning quantities** (Propositions 1 and 2). The inequalities bound how much a visual perturbation can change multi-step prediction error and a planner candidate's squared latent goal cost.
3. **Evaluation across tasks, shifts, and model families** (Section 4). We compare pair-level ACPC with same-horizon controls whose action information is removed, choose joint IR–DR thresholds on source tasks and evaluate them on held-out tasks, and report fixed-severity checks under blur, resize, and PLDM.

2 Related Work

ACPC lies at the intersection of predictive representation learning, robust visual control, and decision-aware model evaluation.

Latent prediction and self-predictive RL. DreamerV3 and TD-MPC2 use learned dynamics to predict the consequences of candidate actions [1, 2]. JEPAs replace pixel reconstruction with prediction in representation space [3, 4, 5, 11]. LeWM brings this design to an end-to-end world model stabilized by SIGReg [10]; its Gaussianity statistic is based on the Epps–Pulley test [12]. PLDM is a separate joint-embedding dynamics design [13, 14].

PBL and SPR are close training-time precedents. PBL predicts future latent observations from histories and action sequences, while SPR rolls an action-conditioned latent transition model toward moving-average targets and uses augmented views in its prediction loss [15, 16]. Later work studies collapse in self-predictive learning and analyzes the action-conditioned objective [17, 18]. These methods use self-prediction to learn a representation. ACPC instead freezes the encoder and predictor, then compares two predicted rollouts under the same actions; its core paired discrepancy does not require an observed future.

Analyses of latent prediction show, under specific assumptions, how predictive learning can favor influential features over noisy ones [6, 7] or recover latent state up to a linear transformation [19]. seq-JEPA conditions prediction on known view transformations and studies the trade-off between invariance and equivariance [20]. Our question is different: after training, does a same-state visual discrepancy contract or grow as the predictor rolls it forward?

Robust visual world models and selective invariance. DrQ and DrQ-v2 improve visual control through training-time augmentation, while SODA uses augmentation in an auxiliary representation-learning objective [21, 22, 23]. ViGMO and VIBR learn invariances in latent or value space [24, 25], while ReOI changes observations at test time to reduce distractor sensitivity in visual model-predictive control [26].

Several methods modify world-model training itself. TPC favors temporally predictable information, and DreamerPro replaces pixel reconstruction with prototype prediction [27, 28]. Task Informed Abstractions and Denoised MDPs use task or reward structure to separate useful state from distractors [29, 30]. Iso-Dream models controllable and noncontrollable dynamics in separate branches, while still using predicted noncontrollable states for value estimation [31]. Other work combines temporal masking with bisimulation objectives, infers actions of visually similar distractors, or uses task-aware reconstruction to focus model capacity [32, 33, 34]. These approaches train a more robust or better-factored world model. ACPC does not change training; it audits a frozen predictor using paired views and shared actions.

Control-aware representation learning also explains why robustness cannot mean removing every difference. Classical bisimulation grounds behavioral similarity in rewards and action-conditioned transition structure [8, 9, 35], while reward-free variants may retain only the transition term [36]. In contrast, value-equivalent and value-aware formulations organize learning around downstream value or planning quantities [37, 38]. These formulations motivate selective invariance, but our diagnostic is narrower: it compares specified visual perturbations and a finite set of logged coordinate labels, without testing reward or transition equivalence.

Decision-aware diagnostics. Objective-mismatch work shows that one-step model likelihood need not track downstream control performance [39]. A later survey organizes methods that align model learning with downstream decisions [40]. A recent position paper argues that a world-model evaluation should match its claimed decision-making use, including interventions, long rollouts, planning, and policy evaluation or optimization [41].

Recent methods probe or enforce different aspects of learned dynamics. ATM compares action information in encoded and predicted transitions [42]; Delta-JEPA decodes actions from latent differences [43]; ACID tests cycle action consistency with inverse dynamics [44]; Future-Compatible asks whether generated futures agree with their stated actions [45]; and World Models as Group Actions tests identity, inverse, and composition structure [46]. MWM uses observation-level action-conditioned rollout consistency as a training objective [47], while other work diagnoses kinematic failures in long rollouts [48]. Frozen inverse-dynamics probes also test whether video-world-model latents retain action information under visual corruptions [49]. These tests examine action content or rollout validity; ACPC asks how a specified same-state visual change propagates through the predictor.

Recent adversarial evaluations measure how visual attacks degrade closed-loop world-model agents. WMAttack searches over attack configurations, whereas ARB4WM separates policy-, value-, and latent-dynamics objectives for Dreamer-family agents [50, 51]. ACPC instead uses matched task-preserving views and reports a checkpoint diagnostic rather than adversarial return degradation. Other concurrent work compares predicted rollouts with environment transitions or compares predicted and true plan costs [52, 53]. Those diagnostics measure model error; ACPC measures the change between two views and can be computed without a true future.

Robustness certificates. CARRL certifies Q-based action choices under bounded observation uncertainty, and CROP uses smoothing to certify per-state actions and finite-horizon reward bounds [54, 55]. Our result is narrower: for an evaluated clean–perturbed pair and an aligned finite candidate pool, the stated margin conditions preserve the winner or the exact top-k elite set. They do not certify the complete ranking, policy actions, or return.

Together, these lines of work motivate a paired, planner-facing audit of a frozen model. ACPC measures the rollout change, IR summarizes upper-tail same-history sensitivity, and DR checks separation on protocol-selected pairs with different logged state-coordinate labels.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A state-preserving visual change can still move the encoded history. The predictor may amplify or contract this shift before the planner computes a cost. We trace the effect at three levels. The local-geometry case study in Section 4.2 compares perturbation clouds with nearby clean histories. It is descriptive because learned-space neighbors follow their own recorded actions and endpoint views omit intermediate predictions. Pairwise ACPC instead holds the actions fixed throughout a clean–perturbed rollout and yields exact bounds on prediction-error and candidate-cost changes. Finally, IR summarizes same-history sensitivity across logged histories, while DR checks that selected state-coordinate distinctions remain separated. Figure 1 contrasts fragile and robust checkpoints through the paired rollout and the two summaries. All three analyses are post-hoc measurements of a frozen model; none requires retraining.

3.1 Pairwise ACPC

ACPC asks a direct question: if only the visual appearance of a history changes, how much does the model's predicted future change under a fixed action sequence? Let h be the clean observation–action history consumed by the encoder of the checkpoint under evaluation, and let $\tilde{h}$ be a visually perturbed view of the same underlying state trajectory. The frozen encoder gives $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both branches receive the same action sequence $\mathbf{a} = (a_0, \dots, a_{H-1})$. For its length-$k$ prefix, define

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \qquad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \tag{1}$$

Here F_θ is the action-conditioned predictor and H is the number of autoregressive future steps. The planner does not read raw predictor outputs: it scores candidates in the projected embedding space in which planner costs are computed. The map Π denotes that projection; our implementation stores the projected embeddings directly, so Π requires no extra computation. With nonnegative weights $\sum_{k=1}^{H} \alpha_k = 1$, stack the whole rollout as

$$\bar{G}_{\mathbf{a}}(z) = \begin{bmatrix} \sqrt{\alpha_1}\Pi(\hat{z}_1)^\top & \cdots & \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top \end{bmatrix}^\top. \tag{2}$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\| \bar{G}_{\mathbf{a}}(E_\theta(h)) - \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) \right\|_2 = \left(\sum_{k=1}^{H} \alpha_k \left\| \Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k) \right\|_2^2 \right)^{1/2}. \tag{3}$$

Encoder shift $\|z - \tilde{z}\|_2$ measures the effect of the visual change before prediction. ACPC measures its effect on the predicted trajectory after both histories have been propagated under the same actions. Holding the actions fixed isolates sensitivity to the visual perturbation; it does not establish whether the model responds correctly to other actions. Unless stated otherwise we use uniform weights $\alpha_k = 1/H$ and the fixed protocol horizon $H = 8$, the longest horizon for which every logged evaluation window contains a complete observed future (Appendix C). The planner analyses in Section 4.5 instead use $H = 5$, matching the five-action planning horizon of the evaluated CEM configuration.

3.2 Common-future error drift

ACPC can be computed without observing the true future. When a logged future is available, however, the same quantity bounds how much the visual perturbation can change prediction error. Let $Y_{\mathbf{a}}^H$ be one actually observed future under the same recorded action sequence, projected, stacked, and weighted exactly as in Equation (2). This target is not produced by either prediction branch and is used only for evaluation. Comparing both branches with this one future isolates the error change caused by the visual perturbation.

Proposition 1 (Common-future error-drift bound)**.** *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_\theta(h)) - Y_{\mathbf{a}}^H\|_2, \qquad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) - Y_{\mathbf{a}}^H\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{4}$$

The adverse increase $(e_{\tilde{h}} - e_h)_+$, where $(x)_+ = \max(x, 0)$, obeys the same bound.

The proof—a direct application of the reverse triangle inequality in the weighted projected-rollout space, with both errors measured against the same $Y_{\mathbf{a}}^H$—is spelled out in Appendix A.

The proposition bounds the difference between two errors, not either prediction's absolute error: both predictions may be inaccurate even when ACPC is small. The prediction experiment in Section 4.4 uses a logged future only to evaluate whether ACPC is informative about the bounded quantity, which we call the *error drift* $d = |e_{\tilde{h}} - e_h|$.

3.3 Candidate-cost drift and planner stability

We next connect ACPC to planning. A visual perturbation moves each candidate's predicted endpoint and therefore may change its cost. If no candidate can move far enough to cross the clean cost gap, the selected candidate remains unchanged. This is the radius–margin argument formalized below. For candidate action sequence $\mathbf{a}^j$, let

$$x_j = \Pi(F_\theta^H(E_\theta(h), \mathbf{a}^j)), \qquad \tilde{x}_j = \Pi(F_\theta^H(E_\theta(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is the final-step component of the candidate-specific $\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)$. If $\alpha_H > 0$, then $r_j \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)/\sqrt{\alpha_H}$; the planner experiments in Section 4.5 record r_j directly.

Proposition 2 (Squared goal-cost drift and fixed-pool stability)**.** *Suppose the planner scores candidate j against fixed goal embedding g by $C_j = \|x_j - g\|_2^2$ and $\tilde{C}_j = \|\tilde{x}_j - g\|_2^2$. Define*

$$b_j = r_j \big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2 \big). \tag{5}$$

Then every candidate satisfies $|\tilde{C}_j - C_j| \le b_j$.

Let w and $\tilde{w}$ be the clean and perturbed winners of the same pool. The clean-reference selection regret obeys

$$0 \le C_{\tilde{w}} - C_w \le b_{\tilde{w}} + b_w. \tag{6}$$

The quantity b_j is a candidate-specific upper bound on the movement of the squared goal cost, and the regret inequality bounds the extra clean-history model cost of choosing the perturbed winner; both use the evaluated endpoints, not a global Lipschitz approximation. Subtracting the allowed movements from each clean candidate gap yields selection certificates: if every remainder is positive, the clean winner—and, under the analogous elite-wise condition, the exact top-k elite set—is preserved (Proposition 3). Combining b_j with the final-step conversion $r_j \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)/\sqrt{\alpha_H}$ gives the explicit chain

$$|\tilde{C}_j - C_j| \;\le\; b_j \;\le\; \frac{\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}} \big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big), \tag{7}$$

Thus candidate-specific ACPC bounds each candidate's cost change. The regret bound and the sufficient conditions for preserving the winner or elite set are given in Equations (6), (15) and (16). Proofs and equality cases are in Appendix A. Because every candidate must be evaluated under both histories, these certificates support post-hoc analysis and are not an online screening method.

The elite-set condition can also be applied iteratively. CEM repeatedly fits its next proposal to the current elite set [56]. If the clean and perturbed runs start from the same proposal and use common random numbers, preserving the elite set at every iteration keeps their pools and proposals aligned and makes their returned actions identical (Corollary 1). The guarantee stops at the first iteration whose elite condition fails, and it concerns candidate selection and model cost, not simulator return. Formal statements and proofs are in Appendix A.

3.4 IR and DR for checkpoint screening

The bounds above evaluate individual history pairs and planner candidates. Checkpoint screening instead requires a summary over many logged histories, and the resulting summaries do not estimate the candidate-wise planner certificate. Low same-history distance alone is not enough: a collapsed or excessively contracted representation can make every paired rollout look stable. We therefore ask two questions, mirroring the two sides of bisimulation-style equivalence (Section 2). The *Invariance Radius* (IR) measures upper-tail same-history sensitivity under the chosen visual shift. The *Distinction Rate* (DR) checks how often nearby histories with different state-coordinate labels remain farther apart than IR plus a fixed margin. Lower IR and higher DR are favorable, and a checkpoint must satisfy both.

Evaluation uses a fixed set of *anchors*. Each anchor consists of one logged history window, its H-step observed future, and its recorded actions. We first normalize ACPC separately for each anchor and then aggregate across anchors. Throughout this section, Q_q denotes the empirical q-quantile, n is the number of anchors, and M is the number of perturbation draws per anchor. Distances for anchor i are normalized by its clean-motion scale s_i: the median of its H consecutive observed clean-frame displacements in the projected space (Equation (23), Appendix C). Thus s_i is the anchor's typical one-step clean motion; dividing by it expresses rollout distances in units of ordinary motion and uses no task outcome. With a small numerical stabilizer $\varepsilon_{\mathrm{norm}}$ (value in Appendix C) and perturbation draw m, define

$$R_i^{(m)} = \frac{\mathrm{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\mathrm{norm}}}, \qquad \bar{R}_i = \frac{1}{M}\sum_{m=1}^{M} R_i^{(m)}. \tag{8}$$

The raw Invariance Radius is

$$\mathrm{IR}_q^{\mathrm{raw}}(\theta) = Q_q\big(\{\bar{R}_i\}_{i=1}^n\big)\,. \tag{9}$$

IR is one number per checkpoint. It reports a high quantile of normalized ACPC and therefore emphasizes histories with relatively high sensitivity to the evaluated shift. We use q90 because a mean can hide a small set of high-radius anchors; the reported reduction is stable across $H \in \{1, 2, 4, 8\}$ and quantiles q80–q95 (Table 1). This summary does not guarantee the tail behavior of planner candidates.

For each eligible anchor, DR selects the nearest history inside a fixed standardized-state neighborhood whose logged future differs in selected task-state coordinates. Both histories are rolled forward under the anchor's actions, so action differences cannot create the measured separation. The same clean-motion normalization used by IR puts the two distances on a common scale. Let D_i^{diff} be this normalized trajectory distance (Equation (24), Appendix C). DR is

$$\mathrm{DR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i \in \mathcal{I}} \mathbf{1}\big[D_i^{\mathrm{diff}} > \mathrm{IR}_q^{\mathrm{raw}}(\theta) + \delta\big]\,, \tag{10}$$

where $\mathcal{I}$ contains anchors with an eligible neighbor carrying a different label. The reported protocol fixes $q = 0.90$ and normalized margin $\delta = 0.10$. DR is the fraction of tested different-label pairs whose rollout distance exceeds IR by more than this margin.

A constant representation makes both IR and every D_i^{diff} zero, so it fails DR. Conversely, a checkpoint can separate the tested labels while remaining visually sensitive, so DR alone does not replace IR. DR is limited to the state-coordinate labels used to construct these pairs. Appendix C gives the exact pairing rules and eligible-pair counts.

3.5 Checkpoint-level threshold selection

Raw IR in Equation (9) is the reference used inside DR. Because raw scales differ across tasks, checkpoint comparisons use relative IR. Let θ_0 be the unaugmented checkpoint for the same task, training run, and model family. Its raw IR is strictly positive in every evaluated setting, so the following ratio is well defined:

$$\widetilde{\mathrm{IR}}_q(\theta;\theta_0) = \frac{\mathrm{IR}_q^{\mathrm{raw}}(\theta)}{\mathrm{IR}_q^{\mathrm{raw}}(\theta_0)}. \tag{11}$$

Raw IR remains in the DR definition; relative IR equals one for the unaugmented reference. This normalization removes a common multiplicative scale factor but does not make raw distances comparable across tasks or model families.

The checkpoint rule uses one threshold for each measure. For model family $\mathcal{F}$ and visual shift υ (e.g., Gaussian noise), let these thresholds be t_I and t_D. Every quantity in the score is measured under the same shift. With a small constant $\varepsilon_{\mathrm{score}}$ (value in Appendix C), define

$$S_{\mathcal{F},\upsilon}(\theta;\theta_0) = \min\left\{ \frac{t_I - \widetilde{\mathrm{IR}}_q(\theta;\theta_0)}{|t_I| + \varepsilon_{\mathrm{score}}}, \frac{\mathrm{DR}_{q,\delta}(\theta) - t_D}{|t_D| + \varepsilon_{\mathrm{score}}} \right\}. \tag{12}$$

The score satisfies $S \geq 0$ exactly when both conditions pass: relative IR is at most t_I, and DR is at least t_D. Planning success on source tasks is used only to choose t_I and t_D; success on held-out tasks is used only to evaluate the fixed rule. Thresholds are chosen separately for each model family. Once the thresholds are fixed, scoring a checkpoint uses no task-success label.

For a comparison with reference checkpoint θ_0, define the paired score change $\Delta S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) = S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) - S_{\mathcal{F},\upsilon}(\theta_0;\theta_0)$. A positive ΔS means only that θ_1 scores better than its reference under the same thresholds; it assigns no absolute robustness label to either checkpoint.

The pair-level experiments test the quantities linked by the two bounds (Sections 4.4 and 4.5). The checkpoint experiments test whether the joint IR–DR rule identifies recovery across tasks, model families, and visual shifts (Sections 4.6 to 4.8). This checkpoint-level result is an empirical screening claim, not a consequence of the pairwise inequalities.

4 Experiments

We first state the evaluation protocol. A geometry case study and the full checkpoint sweep then establish the visual-sensitivity pattern. Next, we test pairwise ACPC against prediction-error drift and CEM selection regret. Finally, we evaluate the IR–DR rule on held-out tasks, PLDM, blur, and resize.

4.1 Evaluation protocol

Models, tasks, and planning evaluation. LeWM [10] is the primary model family, and PLDM [13, 14] tests whether the diagnostic applies to a second joint-embedding dynamics architecture. We evaluate four control tasks: TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. Following the standard LeWM latent-space MPC evaluation, CEM uses the frozen world model to optimize candidate sequences of five actions (plan horizon 5). We report *planning success rate*, the percentage of evaluation episodes that reach the task goal.

We use *visual perturbation* for one sampled change to a history and *visual shift* for the distribution from which that change is drawn. The main evaluation adds Gaussian noise with $\sigma = 0.08$ to the observations while keeping the goal image clean. Additional evaluations use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each model family and task, the Gaussian-noise training sweep contains nine checkpoints: one *unaugmented checkpoint*, trained without noise augmentation, and eight checkpoints trained with full-sequence noise $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$. Each LeWM task–augmentation setting has three training runs (seeds 3072/3073/3074). Each run is evaluated with seeds 42/43/44 and 100 episodes per evaluation seed. We use *task–run cell* for one task and one training run.

The training run is the unit of replication. The three evaluation seeds measure only within-run variability, so the reported means and dispersions across runs are descriptive rather than population estimates. The PLDM sweep has one run for each of its 36 task–augmentation settings, with the same evaluation seeds and episode count.

Diagnostic measurements. IR uses 100 logged anchors, five Gaussian-noise draws at the evaluation severity $\sigma = 0.08$ (blur and resize diagnostics instead apply the corresponding shift), eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. For each anchor, DR finds a nearby history with a different endpoint-state label and checks whether the two rollouts remain farther apart than raw IR plus 0.10. Appendix C gives the exact pairing, normalization, and threshold rules. Checkpoint comparisons use relative IR from Equation (11).

Threshold evaluation. A checkpoint in the Gaussian-noise training sweep meets the *success-rate criterion* if it recovers at least 80% of the improvement from the unaugmented checkpoint to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations. Both constants were fixed a priori. We select (t_I, t_D) on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks. The one-, two-, and three-task selection settings produce $4 + 6 + 4 = 14$ directional threshold-selection/test partitions. Metrics first average training runs within each test task and then weight tasks equally; checkpoint rows are never treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, IR, or DR.

For blur and resize, each task uses the thresholds selected under Gaussian noise on the other three tasks. We compare 24 checkpoint pairs (four tasks $\times$ three runs $\times$ two shifts). Each pair contains the unaugmented checkpoint and the checkpoint trained at $\sigma_{\max} = 0.08$. A pair is positive when success under the shift improves by at least five percentage points and clean success decreases by no more than five points. We make no blur- or resize-specific adjustment. This analysis also does not benchmark alternative signals such as encoder shift or one-step ACPC.

4.2 Do visual perturbations remain local after prediction?

We begin with a PushT case study showing how visual noise can make perturbed views of one history overlap with other histories. For one clean history and its perturbed views, the sampled visual radius r measures the spread of the perturbation cloud; nearest-clean spacing NN is the distance to the nearest other clean history. The ratio r/NN asks whether visual variation overwhelms local between-history spacing, while a symmetric two-ball test counts fully disjoint clouds. These quantities are descriptive: neighbors are chosen in learned space, each clean rollout follows its own recorded actions, and a collapsed representation makes every radius small. They motivate the controlled ACPC comparison but do not replace it.

Using a fixed PushT case study, we compare a matched pair of LeWM checkpoints: one unaugmented and one trained with full-sequence Gaussian-noise augmentation at $\sigma_{\max} = 0.08$. We inspect both the encoder output and the representation after eight autoregressive rollout steps. All ratios and fractions use the original 192-dimensional projected latent space in which the planner computes costs; formal definitions and sampling details are in Appendix B. The t-SNE visualization [57] is qualitative only.

Figure 2 summarizes the case study. Without augmentation, the median r/NN is 1.41 at the encoder and 1.86 after eight rollout steps, and none of the 128 anchor clouds is fully disjoint. After Gaussian-noise augmentation, the corresponding ratios fall to 0.10 and 0.19; the fully disjoint fractions rise to 95.3% at the encoder and 84.4% after the rollout. Thus, in this fixed case study, augmentation makes perturbed views of the same history substantially more compact relative to nearby clean histories. Much of this separation remains after prediction.

This case study shows that augmentation can contract perturbation-induced variation relative to nearby clean histories, including after prediction. It does not establish task-relevant separation or planning robustness. We therefore turn to checkpoint-level IR and DR and their relationship with planning performance across the complete augmentation sweep (Figure 3).

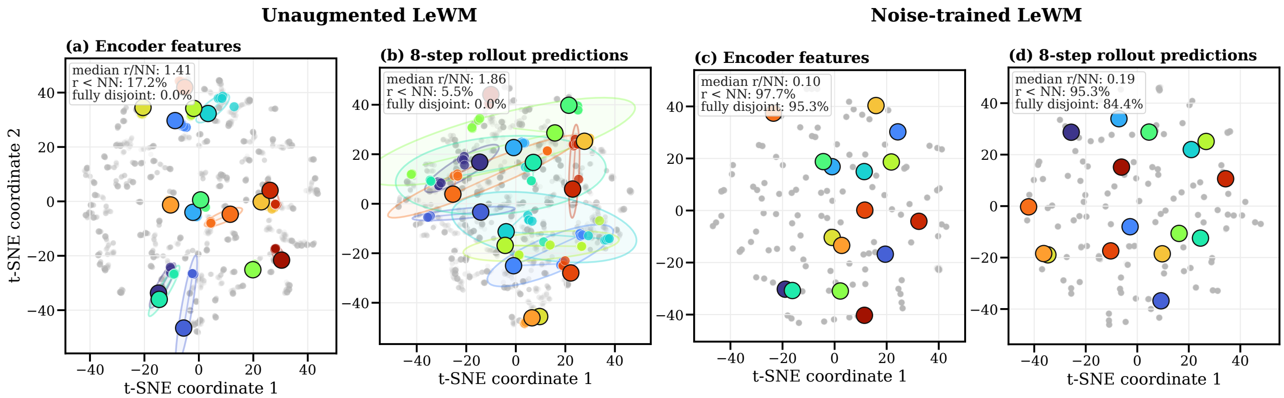

Figure 2: Descriptive local-geometry case study motivating the controlled ACPC comparison. Panels (a)–(d) compare a matched pair of PushT checkpoints—the left two unaugmented, the right two trained with Gaussian-noise augmentation at $\sigma_{\max} = 0.08$—at the encoder output and after eight autoregressive rollout steps. Each color denotes one of 16 highlighted histories: the large black-rimmed marker is its clean anchor, smaller same-color markers are its 18 perturbed views, and the faint ellipse is their 90% t-SNE envelope; gray points show all other histories and views. Under strong contraction, smaller markers and ellipses can lie beneath the clean marker. Each panel's upper-left summary reports, across all 128 anchors, the median r/NN and the percentages with $r < \mathrm{NN}$ and with fully disjoint high-dimensional enclosing balls. The t-SNE coordinates and ellipses are qualitative and enter none of these metrics.

4.3 How do IR and DR change across checkpoint recovery?

At evaluation noise $\sigma = 0.08$, the unaugmented LeWM checkpoints lose 25.9 ± 2.4, 74.6 ± 5.6, 41.0 ± 1.4, and 22.1 ± 2.8 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian-noise augmentation recovers performance over a range of training levels whose location differs by task. Figure 3 compares success rate with IR and DR across the complete sweep.

On Reacher, augmentation also raises clean success from 59.2% to 76–82%. Its recovery under noisy evaluation therefore cannot be attributed to robustness alone; the checkpoints also improve on clean observations.

Every augmentation level that meets the success-rate criterion has lower IR and higher DR than its unaugmented reference, although neither measure is monotone at every level. Across the 108 LeWM checkpoint rows, 77 pass the reported IR threshold $t_I = 0.3$, and all 77 also pass $t_D = 0.95$. Thus every accepted checkpoint combines reduced same-history sensitivity with retention of the evaluated state-coordinate distinctions. The different-label distance medians are largely stable across augmentation levels, so the rise in DR mainly reflects contraction of the same-history radius rather than expansion of different-label distances.

IR tracks the recovery region, while DR verifies that the sampled state-coordinate distinctions remain outside the contracted radius. A first-order estimate of the composed encoder–rollout Jacobian offers one mechanism for the IR reduction (Appendix G). The next two experiments test whether ACPC uses recorded-action information and whether it tracks a planner-facing quantity (Sections 4.4 and 4.5).

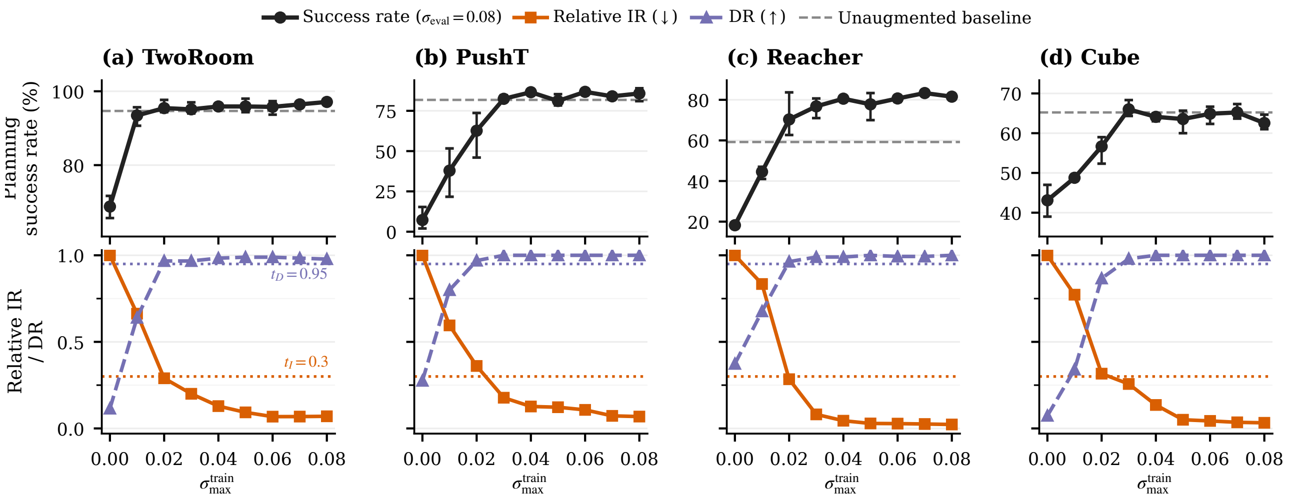

Figure 3: Checkpoint-level planning performance and diagnostics across the complete Gaussian-noise training sweep. We report planning success rate at evaluation noise $\sigma = 0.08$, relative IR, and DR. Points are across-run means; success-rate error bars span the three training runs, and success-rate axes are scaled per task. The dashed line is the clean success rate of the unaugmented checkpoint; the leftmost black point is that checkpoint evaluated at $\sigma = 0.08$. Relative IR equals one there by construction. Lower IR and higher DR are favorable. Dotted lines mark $t_I = 0.3$ and $t_D = 0.95$ (Section 4.6).

4.4 Recorded-action ACPC predicts error drift

Does ACPC help predict how much a visual perturbation changes multi-step prediction error? For each history pair, Proposition 1 shows that the two errors against the same logged future can differ by at most ACPC_H. We test whether measured ACPC is informative about this bounded quantity, the error drift $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor; Table 1 shows that the checkpoint-level conclusions are stable for $H \in \{1, 2, 4, 8\}$.

Data and protocol. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian-noise standard deviations $\sigma \in \{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16, and all rows from one trajectory group stay together. We report the mean absolute error (MAE) on groups excluded from model fitting.

Three nested regressions. Every model contains the probe severity and the encoder-history distance $\|z - \tilde{z}\|_2$.

- **Base** adds one-step ACPC under the recorded action: the information available without a multi-step rollout.
- **Base+Control$_8$** adds the strongest *destroyed-action control*: eight-step ACPC recomputed with *zeroed* actions (every action set to zero), *swapped* actions (the recorded actions of a different trajectory), or *shuffled* actions (the anchor's own actions in permuted temporal order). Each control performs the same eight-step rollout, so it carries rollout length without the recorded action sequence. "Strongest" is an oracle choice: in each task–run cell we report whichever of the three controls attains the lowest test MAE, which makes the comparison conservative.
- **Base+ACPC$_8$** instead adds eight-step ACPC under the recorded actions—the only feature whose action sequence is the one that generated the logged future, and hence the feature matching the right-hand side of Proposition 1.

All eight-step features use uniform weights $\alpha_k = 1/8$ in Equation (3). If Base+ACPC$_8$ improves on Base, multi-step information helps; if it also improves on Base+Control$_8$, the gain is attributable to the recorded actions rather than to merely rolling out eight steps.

Results. Base+ACPC$_8$ attains the lowest cross-validated MAE in all 12 task–run cells (Figure 4). For each training run we average the four task-level relative MAE reductions equally and report the mean $\pm$ sample standard deviation across the three training runs: the reduction is $55.9 \pm 4.7\%$ relative to Base and $51.3 \pm 3.5\%$

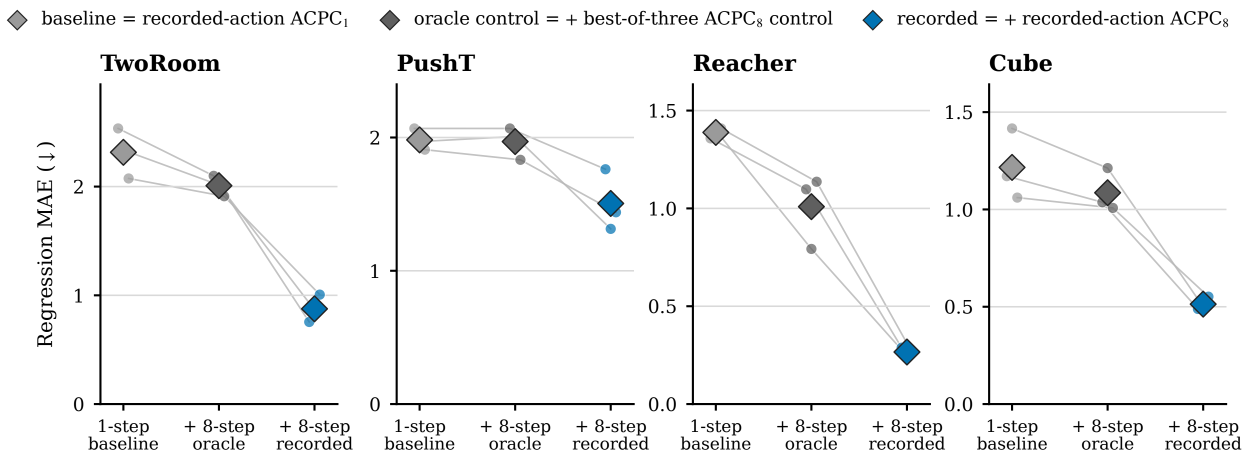

Figure 4: Cross-validated MAE for predicting the error drift d on the unaugmented checkpoints; lower is better. Each model is evaluated on trajectory groups excluded from fitting. Small points are training runs, thin lines join the same run, and diamonds are task means. The in-figure legend labels correspond to Base, Base+Control$_8$, and Base+ACPC$_8$ as defined in Section 4.4. Base+ACPC$_8$ is lowest in all 12 task–run cells; latent-error scales are task-specific, so compare within a panel.

relative to Base+Control$_8$. These results concern prediction of the error drift; they do not compare the absolute forecast accuracy of one-step and eight-step world models. Appendix D reports the task-level values and selected controls (Tables 4 and 5). Eight-step ACPC carries information about the bounded error change that neither encoder shift, one-step ACPC, nor any same-horizon destroyed-action control provides.

4.5 Planner-horizon ACPC predicts CEM selection regret

A prediction change matters to planning only if it affects the plan that CEM selects. The fixed-pool bound motivates the quantity below, but adaptive CEM can generate different later pools once the two elite sets diverge. We therefore evaluate full paired CEM runs rather than treating the fixed-pool condition as a run-level certificate. Our downstream quantity is *clean-reference selection regret.* Let $\mathbf{a}$ and $\tilde{\mathbf{a}}$ be the final plans selected from the clean and perturbed histories, respectively. We measure $(C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a}))_+$: the extra clean-history model cost incurred by selecting the perturbed-history plan. We refer to this single-decision quantity as *CEM selection regret.* It uses the model's squared latent goal cost; it is neither cumulative regret nor simulator return.

The paired CEM branches follow Appendix H: same initial proposal, common random numbers, each branch updating from its own elites. For every candidate, we compute one- and five-step ACPC along its action sequence and take the pool q90; five steps match the planner's prediction horizon. We compare two ridge regressions. The baseline uses perturbation severity, training condition, candidate-level one-step ACPC q90, and the clean gap between the best and second-best candidates. The expanded model adds candidate-level five-step ACPC q90. Within each training run, both regressions are fitted on three tasks and tested on the remaining task, rotating which task is excluded from fitting. We evaluate MAE on $\log(1 + \text{selection regret})$; Appendix H gives the CEM budget and sample counts.

Adding planner-horizon ACPC lowers cross-task test MAE by $15.2 \pm 2.0\%$ (mean $\pm$ sample standard deviation across training runs). The error decreases in all 12 task–run test cases (Figure 5). Since the two regressions differ only by the planner-horizon feature, this gain shows that the planner-horizon rollout carries additional cross-task predictive information. The comparison does not separate action conditioning from horizon length; it tests what the five-step ACPC feature adds. Candidate-specific ACPC therefore helps predict the clean-model cost of a perturbation-induced CEM selection change across tasks. It does not show that ACPC changes the selected action or improves environment performance.

4.6 Do thresholds chosen on some tasks identify recovery on held-out tasks?

Because both the augmentation sweep and the threshold grid are discrete, we test for a useful shared threshold range rather than an exact universal boundary. A decision is correct when it matches the success-rate criterion

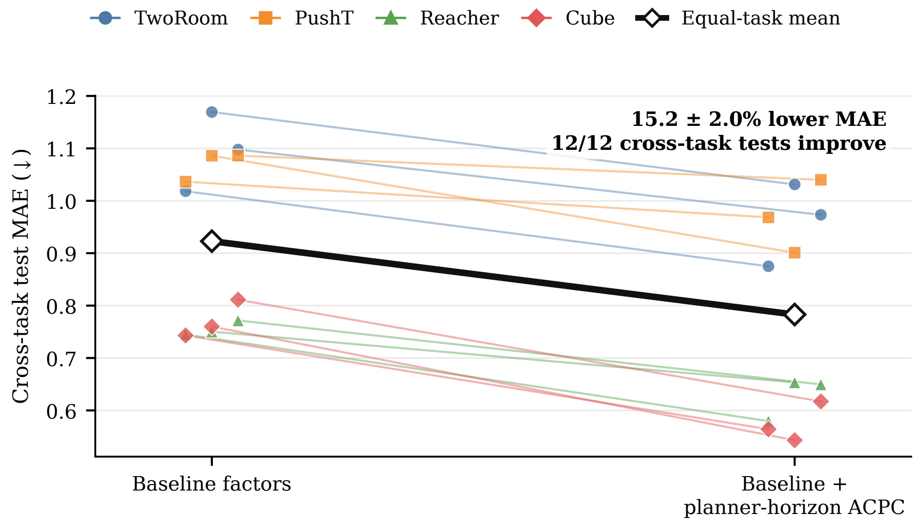

Figure 5: Adding planner-horizon ACPC improves prediction of adaptive-CEM selection regret on tasks excluded from regression fitting. Each colored line connects the baseline and expanded-model errors for one test task in one training run; the thick black line is the equal-task average across the three runs. The added feature is candidate-specific ACPC q90 at the planner's five-step prediction horizon. The baseline already contains severity, training condition, one-step ACPC q90, and the clean best–second-best cost gap. MAE is computed on $\log(1 + \text{selection regret})$.

of Section 4.1. We also count the grid steps between the first accepted checkpoint and the first checkpoint that meets that criterion.

Across the 14 source/test splits, 13 choose $(t_I, t_D) = (0.3, 0.95)$; only the Reacher-only split chooses $(0.1, 0.95)$. With two or three source tasks, the first accepted checkpoint is within 0.5 grid steps of recovery on average and never differs by more than one step (balanced accuracy 0.900, precision/recall 0.913/0.953). Single-source selection is less reliable: balanced accuracy averages 0.855 and ranges from 0.723 to 0.913. Reacher alone chooses the tighter IR threshold and delays acceptance on the other tasks by as many as five levels (Appendix E).

The predominant rule accepts a checkpoint only when relative IR is at most 0.3 and DR is at least 0.95. IR limits same-history sensitivity, while DR requires the tested state-coordinate distinctions to remain separated. Across the sweep, both measures improve over augmentation levels associated with recovery (Figure 3). Because $t_I = 0.3$ is the largest tested value, the upper edge of the useful range remains unresolved.

4.7 Does the diagnostic transfer to PLDM?

We apply the same ACPC, IR, and DR definitions to PLDM, which uses a different architecture and training recipe. The experiment repeats the nine-checkpoint Gaussian-noise training sweep, paired histories, eight-step rollouts, success-rate criterion, and threshold-selection procedure. PLDM has one training run per setting, so run-to-run variation is not measured. Threshold selection stays within PLDM: each test task uses the other three PLDM tasks as sources.

All four leave-one-task-out selections choose $(t_I, t_D) = (0.3, 0.95)$. Acceptance starts at the observed recovery level on PushT and Cube and one level later on TwoRoom. On Reacher, the best noisy score exceeds the unaugmented score by only 1.3 points; one checkpoint meets the relative success criterion, while the joint IR–DR rule accepts none, giving balanced accuracy 0.5. Pooled over all 36 PLDM rows, the rule obtains balanced accuracy 0.836, precision 0.789, and recall 0.882.

The accepted PLDM checkpoints have both low IR and high DR, matching the low-IR, high-DR pattern observed on LeWM. Applying the LeWM grid point after within-task PLDM normalization happens to give the same decisions, but this coincidence does not establish architecture-invariant thresholds. The result shows that the same two-part checkpoint rule can be applied to PLDM; one run per setting limits the evidence.

4.8 Does relative ordering persist under blur and resize?

Blur $(k = 15)$ and resize (scale 0.25) each provide one fixed-severity check outside the Gaussian-noise shift used for threshold selection. For each of the 24 checkpoint pairs in Section 4.1, we recompute IR and DR under the

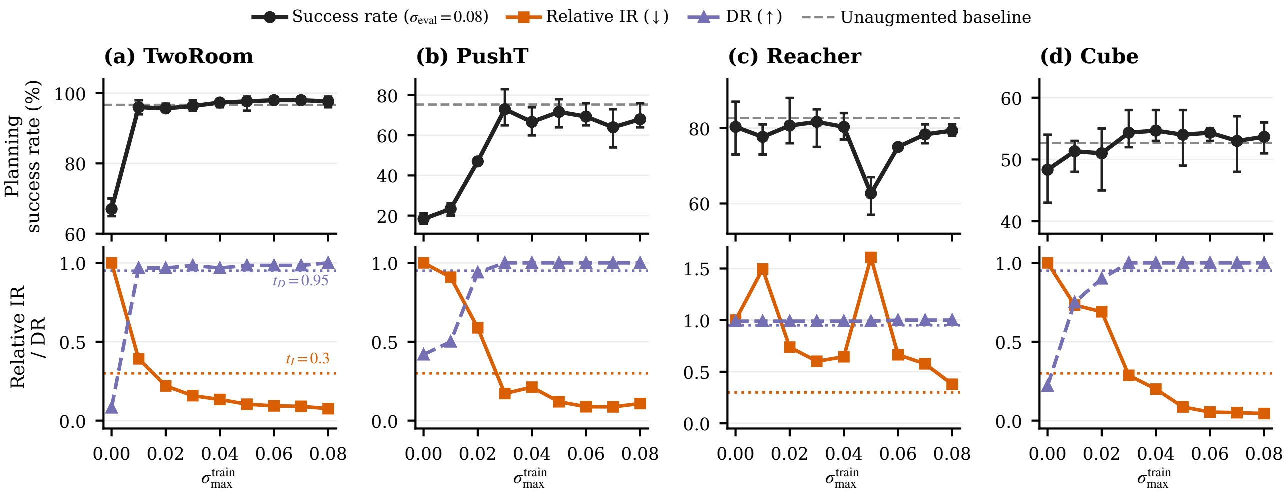

Figure 6: PLDM Gaussian-noise training sweep (one training run per setting). Success-rate error bars span the three evaluation seeds of that run. The dashed gray line marks clean success of the unaugmented checkpoint, and the leftmost black point is its success at evaluation noise $\sigma = 0.08$. Relative IR equals one at that checkpoint. Dotted lines mark $t_I = 0.3$ and $t_D = 0.95$. IR falls below its threshold near the recovery levels on TwoRoom, PushT, and Cube. On Reacher, relative IR remains above t_I, so no checkpoint passes the joint rule.

new shift and compare the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart using ΔS. A positive ΔS means that the augmented checkpoint gets a higher joint IR–DR score. This is a relative comparison, not an absolute pass decision. No blur or resize outcome enters threshold selection.

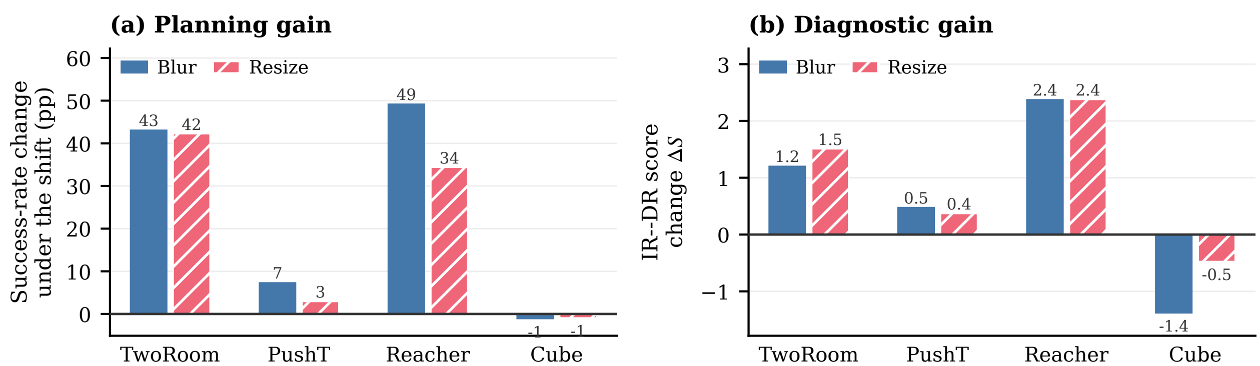

Figure 7: Relative checkpoint comparison under blur and resize. Bars average the three training runs for each task and shift; per-pair values are in Table 9. Panel (a) shows the change in planning success from the unaugmented to the $\sigma_{\max} = 0.08$ checkpoint. Panel (b) shows the change in the IR–DR score of Equation (12); positive favors the augmented checkpoint but is not an absolute pass decision.

The sign of ΔS agrees with whether the augmented checkpoint meets the predefined success criterion on 22 of 24 pairs (balanced accuracy 0.889), and its magnitude has Spearman correlation 0.835 with the success change. The two disagreements are PushT pairs that improve by four percentage points, one point below the criterion. Because ΔS compares two checkpoints, these fixed-severity results support relative ordering, not an absolute robustness decision.

5 Discussion and limitations

What is supported. The strongest evidence is pair-level. Recorded-action ACPC predicts multi-step error drift beyond encoder shift, one-step ACPC, and same-horizon destroyed-action controls. Planner-horizon ACPC also improves prediction of CEM selection regret beyond the corresponding one-step feature and clean candidate margin. These results align with the exact samplewise bounds while testing quantities that remain informative beyond simpler controls.

At the checkpoint level, relative IR tracks recovery across the Gaussian-noise training sweeps, and thresholds chosen on source tasks remain useful on held-out tasks. Under blur and resize, the joint score agrees with the predefined success criterion on 22 of 24 pairs.

Why both measures are necessary. IR alone rewards any contraction, including collapse or excessive invariance. DR guards against that failure by requiring selected histories with different state-coordinate labels to remain separated. Conversely, DR alone does not require two views of the same history to agree. The two measures therefore answer different questions and must be used together. Their scope is limited to the tested visual changes and state-coordinate labels.

Other limits concern the downstream claims. The planner experiment predicts a clean-model cost; it does not intervene on CEM or show improved simulator return. The adaptive-CEM certificate applies only while candidate pools remain aligned and stops at the first failed elite condition. The selected IR point lies at the upper edge of the evaluated grid. LeWM has three training runs per setting, PLDM has one, and blur and resize are each tested at one severity. Finally, the main recovery sweep varies one training intervention: Gaussian-noise augmentation.

Future work. Important next steps are direct task outcomes for ACPC-informed planner interventions, multiple severities per visual shift, richer pairs with different state labels, additional training interventions, and more JEPA-family world models.

6 Conclusion

ACPC measures how much a state-preserving visual change alters a world model's predicted trajectory when the actions are held fixed. Exact inequalities relate this change to prediction-error drift and planner candidate-cost movement. IR summarizes upper-tail same-history sensitivity, while DR checks that nearby histories with different state-coordinate labels remain separated. In our experiments, recorded-action ACPC improves prediction of error drift and CEM selection regret, and joint IR–DR thresholds chosen on source tasks identify recovery on held-out tasks. Results on PLDM, blur, and resize provide initial evidence beyond the primary Gaussian-noise setting.

References

[1] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[2] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[3] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022. Version 0.9.2.

[4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[6] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint-embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 21897–21937, 2025.

[7] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 91300–91336, 2024.

[8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[9] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[10] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[11] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[12] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[13] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. arXiv preprint arXiv:2211.10831, 2022. Self-Supervised Learning: Theory and Practice Workshop at NeurIPS 2022.

[14] Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 43905–43941, 2025.

[15] Zhaohan Daniel Guo, Bernardo Avila Pires, Bilal Piot, Jean-Bastien Grill, Florent Altché, Rémi Munos, and Mohammad Gheshlaghi Azar. Bootstrap latent-predictive representations for multitask reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3875–3886. PMLR, 2020.

[16] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[17] Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Avila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 33632–33656. PMLR, 2023.

[18] Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, Yunhao Tang, Clare Lyle, Mark Rowland, Nicolas Heess, Diana L. Borsa, Arthur Guez, and Will Dabney. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 181–189. PMLR, 2025.

[19] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model? arXiv preprint arXiv:2605.26379, 2026.

[20] Hafez Ghaemi, Eilif B. Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 32943–32973, 2025.

[21] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[22] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[23] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[24] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[25] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[26] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[27] Tung D. Nguyen, Rui Shu, Tuan Pham, Hung Bui, and Stefano Ermon. Temporal predictive coding for model-based planning in latent space. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8130–8139. PMLR, 2021.

[28] Fei Deng, Ingook Jang, and Sungjin Ahn. DreamerPro: Reconstruction-free model-based reinforcement learning with prototypical representations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 4956–4975. PMLR, 2022.

[29] Xiang Fu, Ge Yang, Pulkit Agrawal, and Tommi Jaakkola. Learning task informed abstractions. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3480–3491. PMLR, 2021.

[30] Tongzhou Wang, Simon S. Du, Antonio Torralba, Phillip Isola, Amy Zhang, and Yuandong Tian. Denoised MDPs: Learning world models better than the world itself. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 22591–22612. PMLR, 2022.

[31] Minting Pan, Xiangming Zhu, Yunbo Wang, and Xiaokang Yang. Iso-Dream: Isolating and leveraging noncontrollable visual dynamics in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 23178–23191, 2022.

[32] Ruixiang Sun, Hongyu Zang, Xin Li, and Riashat Islam. Learning latent dynamic robust representations for world models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47234–47260. PMLR, 2024.

[33] Yucen Wang, Shenghua Wan, Le Gan, Shuai Feng, and De-Chuan Zhan. AD3: Implicit action is the key for world models to distinguish the diverse visual distractors. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51546–51568. PMLR, 2024.

[34] Miles Hutson, Isaac Kauvar, and Nick Haber. Policy-shaped prediction: Avoiding distractions in model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 13124–13148, 2024.

[35] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[36] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[37] Christopher Grimm, André Barreto, Satinder P. Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5541–5552, 2020.

[38] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[39] Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pages 761–770. PMLR, 2020.

[40] Ran Wei, Nathan Lambert, Anthony D. McDonald, Alfredo Garcia, and Roberto Calandra. A unified view on solving objective mismatch in model-based reinforcement learning. *Transactions on Machine Learning Research*, 2024.

[41] Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. *arXiv preprint arXiv:2606.15032*, 2026.

[42] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models. arXiv preprint arXiv:2606.09028, 2026.

[43] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding. arXiv preprint arXiv:2606.31232, 2026.

[44] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models. arXiv preprint arXiv:2607.02403, 2026.

[45] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? Diagnosing dynamic consistency in world action models. arXiv preprint arXiv:2605.07514, 2026.

[46] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions. arXiv preprint arXiv:2605.24578, 2026.

[47] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction. arXiv preprint arXiv:2603.07799, 2026.

[48] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure. arXiv preprint arXiv:2607.05966, 2026.

[49] Jewon Yeom, Hanseul Kim, Jeongjae Park, Sungmok Jung, Jaejin Lee, and Taesup Kim. What makes video world model latents action-relevant: Prediction over reconstruction. *arXiv preprint arXiv:2606.07687*, 2026.

[50] Zhixiang Guo, Siyuan Liang, Shi Fu, Cheng Guo, András Balogh, Márk Jelasity, and Dacheng Tao. WMAttack: Automated attack search for adversarial evaluation of world-model agents. *arXiv preprint arXiv:2605.23220*, 2026.

[51] Junjian Zhang, Hao Tan, Ruonan Li, Dong Zhu, Aiping Li, and Zhaoquan Gu. ARB4WM: An adversarial robustness benchmark for world models in continuous control. *arXiv preprint arXiv:2606.16605*, 2026.

[52] Donna Vakalis. Operator-on-F complements value-equivalence: A planning-time diagnostic for latent world models. *arXiv preprint arXiv:2607.04464*, 2026.

[53] Hanzhe You, Yonggang Zhang, Maohao Ran, Zhiqin Yang, Zhenyuan Zhang, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. A control theory of predictability in latent world models. *arXiv preprint arXiv:2607.10362*, 2026.

[54] Björn Lütjens, Michael Everett, and Jonathan P. How. Certified adversarial robustness for deep reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 1328–1337. PMLR, 2020.

[55] Fan Wu, Linyi Li, Zijian Huang, Yevgeniy Vorobeychik, Ding Zhao, and Bo Li. CROP: Certifying robust policies for reinforcement learning through functional smoothing. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[56] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[57] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[58] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In Lise Getoor and Tobias Scheffer, editors, *Proceedings of the 28th International Conference on Machine Learning*, pages 833–840. Omnipress, 2011.

[59] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 18(3):1059–1076, 1989.

A Proofs and Fixed-Pool Analysis

Proof of Proposition 1. Write $u = \bar{G}_{\mathbf{a}}(E_\theta(h))$, $v = \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))$, and $y = Y_{\mathbf{a}}^H$. All three vectors live in the weighted projected-rollout space of Equation (2), and both errors are measured against the same y. The reverse triangle inequality gives

$$|e_{\tilde{h}} - e_h| = \big|\, \|v - y\|_2 - \|u - y\|_2 \,\big| \tag{13}$$

$$\le \|(v - y) - (u - y)\|_2 = \|v - u\|_2 = \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{14}$$

Equality holds exactly when $u - y$ and $v - y$ are positively colinear. The adverse increase obeys the same bound because $(e_{\tilde{h}} - e_h)_+ \le |e_{\tilde{h}} - e_h|$.

Proposition 3 (Fixed-pool selection certificates)**.** *In the setting of Proposition 2, if the clean winner w is unique and*

$$\min_{j \neq w} \big(C_j - C_w - b_j - b_w\big) > 0, \tag{15}$$

then the perturbed branch has the same winner. If $\mathcal{E}$ is the clean top-k elite set and

$$\min_{i \in \mathcal{E},\, j \notin \mathcal{E}} \big(C_j - C_i - b_j - b_i\big) > 0, \tag{16}$$

then the perturbed branch has exactly the same elite set.

Corollary 1 (Conditional CEM stability)**.** *Consider clean and perturbed CEM runs initialized by the same proposal and sampled with common random numbers. While their ordered candidate pools remain aligned, if Equation* (16) *holds at the current iteration, both runs fit the same next proposal. If it holds at every iteration, their pools and proposals remain aligned by induction; since the evaluated solver returns the final proposal mean as its action, the selected actions then coincide as well.*

Proof of Proposition 2. For one candidate, suppress the index and factor the squared-distance difference:

$$
\begin{aligned}
|\tilde{C} - C| &= \big|\|\tilde{x} - g\|_2^2 - \|x - g\|_2^2\big| && (17)\\
&= \big|\langle \tilde{x} - x, \tilde{x} + x - 2g\rangle\big| && (18)\\
&\le \|\tilde{x} - x\|_2 \, \|\tilde{x} + x - 2g\|_2 && (19)\\
&\le r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big) = b. && (20)
\end{aligned}
$$

The first inequality is Cauchy–Schwarz and the second is the triangle inequality. Thus, for nominal winner w and competitor j,

$$\tilde{C}_j - \tilde{C}_w \ge C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \ge C_j - C_w - b_j - b_w.$$

For the perturbed winner $\tilde{w}$,

$$C_{\tilde{w}} \le \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \le \tilde{C}_w + b_{\tilde{w}} \le C_w + b_w + b_{\tilde{w}},$$

which proves Equation (6); nonnegativity follows from the optimality of w on the nominal pool.

For Proposition 3, positivity of Equation (15) makes every perturbed competitor strictly more costly than w. The same argument for every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (16) preserves the exact elite set. Strict inequalities avoid relying on implementation-specific tie breaking.

Proof of Corollary 1. Index the branches by $b \in \{\mathrm{c}, \mathrm{p}\}$ (clean, perturbed) and iterations by t. Let ϕ_t^b collect branch b's proposal parameters (mean and variance), let ω_t be the shared random numbers, and let $\mathcal{A}_t^b = T(\phi_t^b, \omega_t)$ be the sampled candidate pool, where T is the deterministic sampling map. Branch b selects the elite set $\mathcal{E}_t^b$ as the k lowest-cost candidates under its own costs and updates $\phi_{t+1}^b = U(\{\mathbf{a}^j : j \in \mathcal{E}_t^b\})$, where U is deterministic and permutation invariant; the evaluated solver's unweighted elite mean and standard deviation satisfy this. The induction step is

$$\phi_t^{\mathrm{c}} = \phi_t^{\mathrm{p}} \implies \mathcal{A}_t^{\mathrm{c}} = \mathcal{A}_t^{\mathrm{p}} \overset{\text{Equation (16)}}{\implies} \mathcal{E}_t^{\mathrm{c}} = \mathcal{E}_t^{\mathrm{p}} \implies \phi_{t+1}^{\mathrm{c}} = \phi_{t+1}^{\mathrm{p}},$$

and the base case $\phi_0^{\mathrm{c}} = \phi_0^{\mathrm{p}}$ holds by assumption, which aligns pools and proposals for as long as the certificate holds at every completed iteration. The evaluated solver returns the final proposal mean as its action, so aligned proposals return identical actions. An implementation that instead returns the best final candidate additionally requires the top-1 condition Equation (15) at the last iteration. If a certificate fails, equality of the elite sets is unknown; the induction stops and no later shared-pool statement is made.

Relation to the weighted rollout radius. The planner bound uses the displacement at the cost horizon. For the weighted stack in Equation (3), $\sqrt{\alpha_H}\, r_j \le \mathrm{ACPC}_H$ whenever $\alpha_H > 0$. The planner experiments record r_j directly, avoiding the looser $1/\sqrt{\alpha_H}$ conversion. This does not require a future target, but it does require evaluating the candidate under both matched histories.

Sharpness without additional assumptions. Neither principal bound admits a uniformly smaller right-hand side from the available quantities alone. For a unit vector u, common target $Y = 0$, and two stacked predictions au and bu with $a, b \ge 0$, the reverse-triangle bound in Equation (4) holds with equality. Likewise, setting $g = 0$, $x = au$, and $\tilde{x} = bu$ makes $|\tilde{C} - C| = |b - a|(a + b) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2)$.

B Local-Geometry Case-Study Definitions

This section defines the descriptive quantities reported by the case study in Section 4.2. The case study uses 128 logged PushT histories; for each, one clean view and 18 perturbed views are generated—six draws at each probe standard deviation in $\{0.01, 0.04, 0.08\}$. The t-SNE display reproduces the deterministic per-panel projections of the original Appendix visualization.

Let $\mathcal{V} \in \{\mathrm{enc}, \mathrm{roll}\}$ denote either the encoder output or the representation after eight autoregressive rollout steps (the protocol horizon H=8; Section 3.1). For anchor history i, let $v_{i,\mathcal{V}}^{(0)}$ be its clean representation and $v_{i,\mathcal{V}}^{(m)}$, $m = 1, \ldots, M$, the representations of M perturbed views. In the rollout space, the clean and perturbed views of each anchor use the same recorded action sequence. We define the sampled visual radius, nearest-clean spacing, and their ratio as

$$r_{i,\mathcal{V}}^{\text{vis}} = \max_{1 \le m \le M} \left\| v_{i,\mathcal{V}}^{(m)} - v_{i,\mathcal{V}}^{(0)} \right\|_2,$$

$$d_{i,\mathcal{V}}^{\text{NN}} = \min_{j \ne i} \left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2, \qquad \rho_{i,\mathcal{V}}^{\text{local}} = \frac{r_{i,\mathcal{V}}^{\text{vis}}}{d_{i,\mathcal{V}}^{\text{NN}}}. \tag{21}$$

The figures abbreviate $\rho_{i,\mathcal{V}}^{\text{local}}$ as r/NN. When this ratio is below one, every sampled perturbed view of anchor i lies closer to its clean representation than that representation lies to the nearest other clean anchor. A ratio of at least one means that the largest sampled displacement reaches or exceeds this nearest-clean distance. This is a geometric comparison; the nearest clean anchor need not lie across a semantic or task-relevant boundary.

The ratio uses only anchor i's perturbation radius. It does not account for the perturbation radius around the neighboring anchor. We therefore add a symmetric test: the enclosing ball around anchor i is *fully disjoint* if it is separated from the enclosing ball around every other anchor:

$$\left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2 > r_{i,\mathcal{V}}^{\text{vis}} + r_{j,\mathcal{V}}^{\text{vis}} \quad \text{for every } j \ne i. \tag{22}$$

Across the sampled anchors, we report three summaries: the median r/NN ratio; the fraction satisfying $r_{i,\mathcal{V}}^{\text{vis}} < d_{i,\mathcal{V}}^{\text{NN}}$; and the fraction satisfying the fully-disjoint condition in Equation (22). The second is a one-sided nearest-center test, whereas the third is a symmetric certificate for the enclosing balls in the original high-dimensional representation. It is unrelated to whether the qualitative t-SNE display ellipses overlap. Because these radii are estimated from sampled views, their values depend on the number of perturbation draws, anchor density, and representation scale. Encoder and final-step comparisons also omit intermediate predictions. Neither summary bounds prediction-error or planner-cost changes; those links are provided by pairwise ACPC in Section 3.1.

C Evaluation and Diagnostic Protocol

This appendix specifies the fixed evaluation and diagnostic protocol used in the main tables.

Evaluation grid. LeWM is evaluated on PushT, TwoRoom, Reacher, and Cube with evaluation seeds 42/43/44 and 100 episodes per seed. The main evaluation perturbs observation pixels with Gaussian noise at standard deviation 0.08 while keeping the goal image clean. Checkpoints trained with noise use full-sequence input-side Gaussian-noise augmentation with $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$.

PLDM protocol. PLDM uses the same four tasks, nine-checkpoint Gaussian-noise training grid, evaluation seeds, trajectory count, paired-history construction, $H = 8$ rollout statistic, and DR definition. For cross-task analysis, we divide raw IR by the value of the corresponding unaugmented PLDM checkpoint. We then select thresholds on the other three PLDM tasks. Success rates from the evaluation task never enter threshold selection. A second test applies the corresponding LeWM threshold pair after the same within-task PLDM normalization. We do not assume that model families share raw thresholds.

Success-rate criterion. Within each task and training run, let P_{base} and P_{best} be the success rates at evaluation noise $\sigma = 0.08$ for the unaugmented checkpoint and the best checkpoint in the sweep. A checkpoint meets the success-rate criterion when its noisy-observation success rate is at least $P_{\text{base}} + 0.8(P_{\text{best}} - P_{\text{base}})$ and its clean success rate is no more than five percentage points below that of the unaugmented checkpoint. Statements about recovered levels in Figure 3 require this criterion to hold in a majority of runs.

IR protocol. We compute IR separately for each task, training run, and checkpoint. For anchor i, let $u_{i,0}^{\text{obs}}$ be the embedding of the last clean history observation in the projected space selected by Π, and $u_{i,1:H}^{\text{obs}}$ the embeddings of the next H actually observed clean frames in that space; the clean-motion scale is

$$s_i = Q_{0.50}\Big(\big\{ \| u_{i,k}^{\text{obs}} - u_{i,k-1}^{\text{obs}} \|_2 \big\}_{k=1}^{H} \Big). \tag{23}$$

Table 2: Implemented state-coordinate labels for DR. The state is taken at the eighth observed future step and standardized coordinate-wise with full-dataset statistics. Counts give eligible and skipped anchors in the fixed 100-anchor evaluation; these labels are not semantic or action-relevance annotations.

Task	Logged state field (dim.)	Implemented label $\ell(y)$	Eligible / skipped
TwoRoom	`pos_agent` (2)	One median bit from the standardized agent x coordinate (slice `pos_agent[0:1]`).	61 / 39
PushT	`state` (7)	Three median bits from standardized block x, block y, and block angle (slice `state[2:5]`).	98 / 2
Reacher	`observation` (6)	Three median bits from the two standardized joint-velocity coordinates and their Euclidean norm (slice `observation[4:6]`).	100 / 0
Cube	`observation` (28)	Three median bits from the first two coordinates and the Euclidean norm of $r = \bar{o}_{0:3} - \bar{o}_{25:28}$, where $\bar{o}$ is the standardized observation.	100 / 0

Each anchor contributes one weighted radius over the rollout horizon. Perturbation draws use Gaussian noise on observations at $\sigma = 0.08$, matching the behavioral evaluation severity; under blur or resize, draws apply that shift instead. The default uses $H = 8$ and uniform weights $\alpha_k = 1/H$. We divide each radius by the anchor's q50 clean observed-transition scale, including the transition from the final history observation to the first future observation. We then average multiple noise draws within each anchor and take q90 across anchors. This gives raw IR in Equation (9), which DR uses as its same-history reference. For sweep plots and cross-task threshold selection within each model family, we divide raw IR by the corresponding unaugmented value as in Equation (11) before computing thresholds or run summaries. The numerical stabilizers are $\varepsilon_{\text{norm}} = 10^{-8}$ in Equation (8) and $\varepsilon_{\text{score}} = 10^{-12}$ in Equation (12). Table 1 reports how the checkpoint-level IR reduction depends on the rollout horizon and the summary quantile.

Table 1: Horizon and quantile sensitivity of the IR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the IR of the checkpoint trained at the highest augmentation level ($\sigma_{\max} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

	$q = 0.90$ by horizon				$H = 8$ by quantile		
Task	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

DR protocol. DR follows Equation (10). Let $d_{0.35}$ be the q35 of all off-diagonal Euclidean distances among the standardized endpoint states; this radius, well below the median pairwise distance, keeps every selected pair genuinely nearby in state space while leaving most anchors an eligible neighbor with a different label (Table 2 reports the counts). Each eligible anchor chooses the nearest neighbor $j(i)$ with a different label and distance at most $d_{0.35}$. The predictor rolls both clean histories under the anchor actions $\mathbf{a}_i$; for the neighbor these are not the actions it actually executed, but reusing the anchor's actions keeps a single shared action sequence on both sides. The normalized distance between the two labeled histories is

$$D_i^{\text{diff}} = \frac{\left\|\bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)}))\right\|_2}{s_i + \varepsilon_{\text{norm}}}. \tag{24}$$

The pair passes only if this distance is strictly greater than raw q90 IR plus 0.10.

Table 2 makes the pairing rule explicit. The logged state is taken at the end of the observed eight-step future and standardized coordinate-wise by the dataset preprocessor. The neighborhood radius is the $d_{0.35}$ defined above, computed in that task's full standardized state vector. State-coordinate labels use median splits computed within the fixed 100-endpoint anchor batch (anchor seed 9101); each label therefore describes the endpoint reached by that trajectory's own recorded actions. After selecting a neighbor, we roll both starting histories under the anchor's actions for the latent-distance test. We skip an anchor only when no state with a different label lies inside the neighborhood. Pairing depends only on the logged states, so the eligible counts remain constant across checkpoints and training runs.

Table 3: Summary of the Gaussian-noise training sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 3 shows every level). "Best" is the level with the highest mean planning success at evaluation noise $\sigma = 0.08$; arrows give the change from the unaugmented checkpoint to that level. Relative IR is lower-is-better, and DR is higher-is-better. The last column lists levels meeting the success-rate criterion (Section 4.1).

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative IR	DR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

Table 4: Relative reduction in out-of-group MAE from Base+ACPC$_8$ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.4). Base+Control$_8$ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC$_8$ is best.

Training run (seed)	vs. Base	vs. Base+Control$_8$	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean ± SD	55.9 ± 4.7%	51.3 ± 3.5%	12/12

Threshold grids. For cross-task evaluation, candidate thresholds are $t_I \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}$ for task-relative IR and $t_D \in \{0.80, 0.85, 0.90, 0.95\}$ for DR. Within each source-task subset, selection minimizes, in order, mean absolute recovery-onset mismatch, false-early count, and false-late count. It then maximizes balanced accuracy, prefers smaller t_I, and finally prefers larger t_D. The selected pair is applied unchanged to every held-out task. No held-out-task label or blur/resize outcome enters threshold selection.

Reproducibility. The scripts documented in `paper1/scripts/README.md` rebuild all summary figures and tables deterministically. Checkpoint-level diagnostics also require the released checkpoints and datasets. Machine-readable summaries record training seeds, evaluation seeds, and protocol hashes.

D Prediction-Error Scope and Controls

The common-future inequality in Proposition 1 has zero numerical violations in any logged eight-step or candidate-specific five-step task×run×training-condition cell; this is an implementation invariant, not an independent statistical success count.

Logged-future prediction and candidate planning are different estimands: the former predicts the error drift against an independent action-matched observed future, whereas the latter compares candidates from the actual planner pool. We do not infer one result from the other; Section 4.5 supplies the planner-facing evidence with same-pool, candidate-specific features.

The prediction-error analysis is restricted to the unaugmented checkpoints and the listed visual probes. Table 4 reports the per-run relative reductions, and Table 5 the underlying out-of-group MAE values with the selected controls. Every training run improves on all four tasks.

E Cross-Task Threshold Partitions

The 14 rows below are exhaustive: selecting one, two, or three of four tasks as sources creates $4+6+4$ directional partitions. Each evaluation task retains every training run and all nine checkpoints. The apparent sample size is therefore not inflated by treating checkpoint rows as independent observations.

The single-source Reacher exception in Section 4.6 is a tie-break in fit: under $(0.3, 0.95)$ Reacher's own onset mismatch is also at most one level, but calibrating on Reacher alone prefers the tighter $t_I = 0.1$ that fits it exactly and does not carry over. Recovered Reacher checkpoints reach relative IR well below 0.1, while the earliest recovered levels on the other tasks lie above it, so the tighter threshold rejects them and delays the accepted onset.

On the PLDM sweep, the most visible single-run artifact is Reacher at $\sigma_{\max} = 0.05$, where the clean and the noisy score drop together. The diagnostic tracks this dip: the two levels with the lowest clean scores ($\sigma_{\max} = 0.01$

Table 5: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.4. "Selected control" is the destroyed-action control chosen by the per-cell oracle; "paired wins" counts the folds (of 16) in which Base+ACPC$_8$ beats Base+Control$_8$.

Task	run (seed)	Base	Base +Control$_8$	Base +ACPC$_8$	selected control	paired wins
TwoRoom	3072	2.535	2.099	0.755	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	0.866	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	1.006	swapped actions	13/16
PushT	3072	2.068	2.068	1.762	shuffled actions	11/16
PushT	3073	1.969	2.008	1.315	zeroed actions	13/16
PushT	3074	1.909	1.833	1.439	zeroed actions	13/16
Reacher	3072	1.358	1.097	0.288	shuffled actions	16/16
Reacher	3073	1.399	0.793	0.247	shuffled actions	16/16
Reacher	3074	1.409	1.136	0.263	shuffled actions	16/16
Cube	3072	1.171	1.037	0.489	zeroed actions	14/16
Cube	3073	1.416	1.212	0.500	zeroed actions	14/16
Cube	3074	1.061	1.008	0.552	shuffled actions	14/16

and 0.05) are exactly the two with the highest relative IR in Figure 6(c), consistent with relative IR responding to checkpoint quality itself rather than only to the augmentation level.

For PLDM, Table 7 lists the per-task balanced accuracies of the leave-task-out screen of Section 4.7 next to the LeWM values; pooled over all 36 PLDM decisions, balanced accuracy is 0.836 with precision 0.789 and recall 0.882.

F Cross-Stressor Pairs and Discordant Cases

Both discordant rows lie near the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the five-point criterion, while their IR–DR scores increase. The complete table shows these cases rather than only the aggregate classification metrics.

G Local Sensitivity to Gaussian Noise as a Mechanistic Interpretation

For a small isotropic observation perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$, first-order expansion of encoder E followed by rollout map G gives $\Delta G = J_G J_E\, \delta x + o(\|\delta x\|_2)$ with Jacobians evaluated at the clean input. Writing $A = J_G J_E$ and taking expectations of the leading term,

$$\mathbb{E}\big[\|A\,\delta x\|_2^2\big] = \operatorname{tr}\!\big(A\,\mathbb{E}[\delta x\,\delta x^\top]A^\top\big) = \sigma^2 \operatorname{tr}(AA^\top) = \sigma^2\|A\|_F^2, \tag{25}$$

so $\mathbb{E}[\|\Delta G\|_2^2] \approx \sigma^2 \|J_G J_E\|_F^2$ to first order as $\sigma \to 0$. Jacobian Frobenius norms have previously been used to quantify and regularize local representation sensitivity and contraction [58]. We estimate $\operatorname{tr}[(J_G J_E)^\top (J_G J_E)] = \|J_G J_E\|_F^2$ with Rademacher probes using the Hutchinson estimator [59]; JVPs avoid materializing the full Jacobian. This explains one mechanism by which training with Gaussian noise can contract ACPC: it reduces the composed local sensitivity of the encoder–predictor path (Figure 8). It is a local approximation, not a global robustness bound.

H Adaptive-CEM Selection-Regret Protocol

The selection-regret analysis uses no task-success labels. It evaluates the unaugmented and $\sigma_{\max} = 0.08$ checkpoints on 100 histories per task, training run, and training condition at perturbation severities $0.02, 0.05, 0.08$. Each CEM branch uses $K = 64$ candidates, eight elites, eight iterations, and a five-step model-prediction horizon. The clean and perturbed branches start from the same proposal and use common random numbers, but each branch updates its proposal from its own elites. The implemented cost is the mean-reduced squared error over embedding coordinates, i.e., $\|x - g\|_2^2/d$ for embedding dimension d; this common positive factor rescales every C_j and b_j identically and leaves the regret bound and both certificates unchanged.

Table 6: Joint IR/DR checkpoint screening with thresholds chosen on other tasks. Thresholds are selected on the threshold-selection tasks and applied unchanged to all test tasks. Metrics first average training runs within each test task and then weight tasks equally. Recovery-onset mismatch is measured in training-augmentation grid levels (one level is 0.01 in $\sigma_{\max}$).

Selection tasks	Test tasks	t_I	t_D	BA	P / R	Onset mismatch mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

Table 7: The same leave-task-out IR–DR screen in two model families: for each evaluation task, thresholds are selected on that family's other three tasks and applied unchanged. Chance is 0.5; LeWM values average three training runs, whereas PLDM has one run per setting. Applying the LeWM threshold pair to PLDM after within-task IR normalization yields identical PLDM decisions.

	Balanced accuracy	
Evaluation task	LeWM	PLDM
TwoRoom	0.935	0.938
PushT	0.917	0.750
Reacher	0.889	0.500
Cube	0.858	0.875

For every candidate, we compute ACPC along that candidate's action sequence and summarize the pool by its q90 at horizons one and five. For clean plan $\mathbf{a}$ and perturbed-history plan $\tilde{\mathbf{a}}$, the prediction target is the clean-reference selection regret

$$\left[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})\right]_+.$$

Within each training run, ridge regressions are fitted on three tasks and evaluated on the fourth. MAE is computed on the log-transformed target and averaged equally across the four tasks excluded from fitting in turn. The reported relative MAE decrease is computed within each run before taking the mean and sample standard deviation across the three runs. This protocol evaluates prediction of a clean-model cost; it does not evaluate simulator return.

Table 8: IR–DR scores under blur and resize. For each task, thresholds are chosen on the other three Gaussian-noise tasks and then fixed; each pair compares the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart. "Discordant" counts pairs whose score-change sign disagrees with the predefined success criterion (Section 4.8).

Visual shift	Pairs	BA	Precision / recall	Spearman	Discordant
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Table 9: All 24 LeWM checkpoint pairs evaluated under blur and resize. $\mathrm{IR}_{\mathrm{rel}}$ and DR are measured for the checkpoint trained with Gaussian-noise augmentation ($\sigma_{\max} = 0.08$); ΔP and ΔS are its success-rate and IR–DR score changes relative to the unaugmented checkpoint. A pair is positive when $\Delta P \geq 5$ percentage points with at most a five-point clean loss. Daggers mark the two discordant pairs.

Task	Seed	Shift	$\mathrm{IR}_{\mathrm{rel}}$	DR	ΔP	ΔS	Outcome (success / IR–DR)
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	positive / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	positive / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	positive / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	positive / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	positive / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	positive / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	positive / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	nonpositive / positive†
PushT	3073	blur	0.846	0.918	7.3	0.515	positive / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	positive / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	nonpositive / positive†
PushT	3074	resize	1.173	0.939	-19.7	-0.577	nonpositive / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	positive / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	positive / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	positive / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	positive / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	positive / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	positive / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	nonpositive / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	nonpositive / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	nonpositive / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	nonpositive / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	nonpositive / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	nonpositive / nonpositive

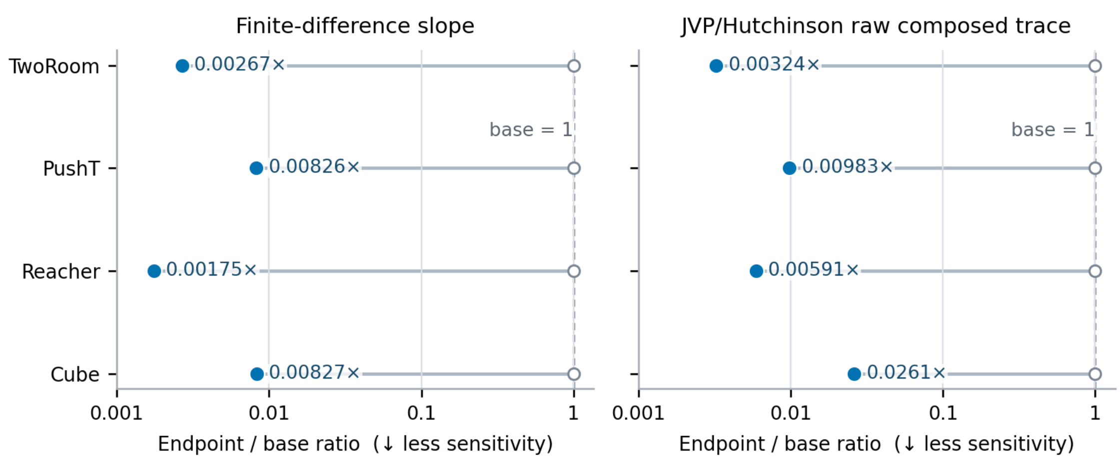

Figure 8: Local-sensitivity ratio between the Gaussian-noise-augmented and unaugmented checkpoints, estimated by finite differences and a JVP-based Hutchinson estimator of the composed Jacobian's squared Frobenius norm. For every task, both ratios are below one, indicating lower local sensitivity after augmentation but not establishing task performance.